

There are three main applications involved in backup and restore operations—`pg_dump`, `pg_dumpall`, and `pg_restore`. As you can imagine from their names, `pg_dump` and `pg_dumpall` are related to extracting (dumping) the content of a database, thus creating a backup, while `pg_restore` is their counterpart and allows you to restore an existing backup.



It is important to note that the tool version matters, and you should always use the tools that match the same PostgreSQL major version. While you can use tools from the most recent version to perform the backup on older clusters, the reverse is not true.

It is important to note that, unlike other database engines, PostgreSQL does not require a special “backup” permission to dump the content of a database: it does suffice that the user executing the backup has the required grants to access the data they want to backup. Similarly, in order to restore data, the user must obtain sufficient permission to write data into tables. However, in order to simplify the permission management, PostgreSQL provides two particular predefined roles — `pg_read_all_data` and `pg_write_all_data`. These roles can be granted to a user who needs to perform a backup and/or a restore, and they will automatically provide the user with all the rights required to access (read) or restore (write) the data.

The `pg_dump` application is used to dump a single database within a cluster, `pg_dumpall` provides us with a handy way to dump all the cluster content, including roles and other intra-cluster objects, and `pg_restore` can handle the output of the former two applications to perform a restoration.



Remember that a backup is valid if, and only if, it can be restored. `pg_dump` and `pg_dumpall` will not produce a corrupted backup, but your storage could accidentally damage your backup files, so to ensure you have a valid backup, you should always try to restore it on another machine or cluster.

All three commands can work locally or remotely on the cluster to backup or restore data, which means you can use them from a remote backup machine or on the same server that the cluster is running on. The applications follow the same parameter and variable conventions that `psql` does, so for instance, you can specify the username that will perform the backup (or restore) via the `-U` command-line flag, as well as the remote host on which the cluster is running via `-h`, and so on. If no parameters are provided, the application assumes the cluster is running locally and connects to it via the current operating system user, just like `psql` does.

In the next subsections, you will learn how to back up and restore your own databases.

Dumping a single database

In order to dump – that is, to create a backup copy of – a database, you need to use the `pg_dump` command.

`pg_dump` allows the following main backup formats to be used, with only the first one suitable for restoration without `pg_restore`:

- **A plain text format:** Here, the backup is made of SQL statements that are reproducible, and it is based on text SQL statements. The resulting backup can be in plain text, or compressed on the fly.
- **A directory format:** The backup is placed into a specific directory, and every database table and large object is placed into a compressed file.
- **A custom format:** This is a PostgreSQL-specific format suitable for a selective restore by means of `pg_restore`.
- **A tar format:** The tar (1) version of the directory format above.

By default, `pg_dump` uses the plain text format, which produces SQL statements that can be used to rebuild the database structure and content, and outputs the backup directly to the standard output. This means that if you back up a database without using any particular option, you will see a long list of SQL statements:

```
$ pg_dump forumdb
-- PostgreSQL database dump
...
SET client_encoding = 'UTF8';
SET standard_conforming_strings = on;
SELECT pg_catalog.set_config('search_path', '', false);
...
CREATE SCHEMA forum;

ALTER SCHEMA forum OWNER TO forum;

SET default_tablespace = '';

SET default_table_access_method = heap;

--
```

```
-- Name: categories; Type: TABLE; Schema: forum; Owner: forum
--

CREATE TABLE forum.categories (
    pk integer NOT NULL,
    title text NOT NULL,
    description text

);

...
COPY forum.tags (pk, tag, parent) FROM stdin;
1      Operating Systems      \N
2      Linux      1
3      Ubuntu      2
4      Kubuntu      3
5      Database      \N
6      Operating Systems      \N
\.

...
```

As you can see, `pg_dump` has produced a set of ordered SQL statements that, if pushed to an interactive connection, allow you to rebuild not only the database structure (tables and functions) but also its content (data within tables), as well as permissions (grants and revokes) and other required objects. All lines beginning with a double dash are SQL comments that `pg_dump` has diligently placed to help you analyze and understand the database's backup content.

There are a few important things to note related to the backup content. The first is that `pg_dump` places a bunch of SET statements at the very beginning of the backup; such SET statements are not mandatory for the backup, but they are needed to restore from this backup's content. In other words, the first few lines of the backup are not related to the content of the backup but, instead, to how to use such a backup.

An important line among those SET statements is the following one, which has been introduced in recent versions of PostgreSQL:

```
SELECT pg_catalog.set_config('search_path', '', false);
```

Such lines remove (i.e., make empty) the `search_path` variable, which is the list of schema names used to search for an unqualified object. The effect of such a line is that every object that's created from the backup during a restore will not exploit any malicious code that could have tainted your environment and your `search_path`. The side effect of this, as will be shown later on, is that after restoration, the user will have an empty search path and will not be able to find any not fully qualified objects by their names.

Another important thing about the backup content is that `pg_dump` defaults to using COPY as a way to insert data into single tables. COPY is a PostgreSQL command that acts like INSERT, allowing multiple tuples to be specified at once, and most notably, it is optimized for bulk loading, resulting in a faster recovery. However, this can make the backup not portable across different database engines, so if your aim is to dump database content in order to migrate it to another engine, you have to specify `pg_dump` to use regular INSERT statements by means of the `--insert` command-line flag:

```
$ pg_dump --insert forumdb

...
INSERT INTO forum.tags OVERRIDING SYSTEM VALUE VALUES (1, 'Operating
Systems', NULL);
INSERT INTO forum.tags OVERRIDING SYSTEM VALUE VALUES (2, 'Linux', 1);
INSERT INTO forum.tags OVERRIDING SYSTEM VALUE VALUES (3, 'Ubuntu', 2);
INSERT INTO forum.tags OVERRIDING SYSTEM VALUE VALUES (4, 'Kubuntu', 3);
INSERT INTO forum.tags OVERRIDING SYSTEM VALUE VALUES (5, 'Database',
NULL);
INSERT INTO forum.tags OVERRIDING SYSTEM VALUE VALUES (6, 'Operating
Systems', NULL);

...
```

The entire content of the backup is the same, but this time, the tables are populated by standard INSERT statements. As you can imagine, the end result is more portable but also longer (and, therefore, bigger in size). However, note how, in the previous example, the INSERT statements did not include the list of columns every field value maps to; it is possible to get a fully portable set of INSERT statements by replacing the `--inserts` option with `--column-inserts`:

```
$ pg_dump --column-inserts forumdb
...
INSERT INTO forum.tags (pk, tag, parent) OVERRIDING SYSTEM VALUE VALUES
(1, 'Operating Systems', NULL);
INSERT INTO forum.tags (pk, tag, parent) OVERRIDING SYSTEM VALUE VALUES
(2, 'Linux', 1);
INSERT INTO forum.tags (pk, tag, parent) OVERRIDING SYSTEM VALUE VALUES
(3, 'Ubuntu', 2);
INSERT INTO forum.tags (pk, tag, parent) OVERRIDING SYSTEM VALUE VALUES
(4, 'Kubuntu', 3);
INSERT INTO forum.tags (pk, tag, parent) OVERRIDING SYSTEM VALUE VALUES
(5, 'Database', NULL);
INSERT INTO forum.tags (pk, tag, parent) OVERRIDING SYSTEM VALUE VALUES
(6, 'Operating Systems', NULL);
...
```

Being able to dump the database content is useful, but being able to store such content in a file is much more useful and allows for restoration to occur at a later date. There are two main ways to save the output of `pg_dump` into a file. One requires that we redirect the output to a file, as shown in the following example:

```
$ pg_dump --column-inserts forumdb > backup_forumdb.sql
```

The other (suggested) way is to use the `pg_dump -f` option, which allows us to specify the filename that the content will be placed in. Here, the preceding command line can be rewritten as follows:

```
$ pg_dump --column-inserts -f backup_forumdb.sql forumdb
```

This has the very same effect as producing the `backup_forumdb.sql` file, which contains the same SQL content that was shown in the previous examples.

`pg_dump` also allows for verbose output, which will print what the backup performs while it is doing so. The `-v` command-line flag enables this verbose output:

```
$ pg_dump -f backup_forumdb.sql -v forumdb
pg_dump: last built-in OID is 16383
pg_dump: reading extensions
...
pg_dump: creating SCHEMA "forum"
pg_dump: creating TABLE "forum.categories"
pg_dump: creating SEQUENCE "forum.categories_pk_seq"
pg_dump: creating TABLE "forum.delete_posts"
pg_dump: creating TABLE "forum.j_posts_tags"
pg_dump: creating TABLE "forum.new_categories"
pg_dump: creating TABLE "forum.posts"
pg_dump: creating SEQUENCE "forum.posts_pk_seq"
...
...
```

Once you have your backup file ready, you can restore it easily. We'll learn how to do this in the next section.

Restoring a single database

If the backup you have produced is plain SQL, you don't need anything other than a database connection to restore it – you can execute a bunch of statements in the correct order to recreate the database content.



`pg_dump` is smart enough to figure out the correct order in which tables and other objects have to be dumped, ensuring their dependencies can be restored in the right order.

It is important to note that, by default, `pg_dump` does not issue, in its backup content, a `CREATE DATABASE` statement. In fact, let's say we produce a backup file as follows:

```
$ pg_dump --column-inserts -f backup_forumdb.sql forumdb
```

The created `backup_forumdb.sql` file will not include any instructions on how to create a new database. This can be handy, but also dangerous: it means that the restoration will happen within the database you are connected to.

Let's assume that we want to restore the database content to another local database, which we will name `forumdb_restore`. Here, the first step is to create a database, as follows:

```
$ psql -c 'CREATE DATABASE forumdb_restore WITH OWNER forum;'
CREATE DATABASE
```

Now, it is possible to connect to the target database and ask `psql` to execute the entire content of the backup file:

```
$ psql -U forum forumdb_restore
forumdb_restore=> \i backup_forumdb.sql
SET
SET
SET
SET
CREATE SCHEMA
ALTER SCHEMA
SET
SET
CREATE TABLE
ALTER TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
ALTER TABLE
INSERT 0 1
```

```
INSERT 0 1
INSERT 0 1
...
```

You will see a list of command output codes such as `INSERT 0 1`, which means a single `INSERT` happened, as well as confirmation of the occurrence of `ALTER TABLE`, `GRANT`, and every other command the backup contains. Depending on the size of the backup, as well as the performance of the machine, the restoration could take from a few seconds to minutes.

Once the restore has completed, it is possible to test whether the backup has been restored – for example, by querying a single table for its data:

```
forumdb_test=> SELECT * FROM tags;
ERROR:  relation "tags" does not exist
LINE 1: SELECT * FROM tags;
                  ^
```

Hold on – this does not mean that the backup and restore process didn't work properly! Remember that `pg_dump` has inserted an appropriate instruction to remove every entry from `search_path`, so `psql` doesn't know how to look up a table named `tags`, while it can regularly find a table with a fully qualified name such as `forum.tags`:

```
forumdb_restore=> SELECT * FROM forum.tags;
pk |          tag          | parent
----+-----+-----
 1 | Operating Systems |
 2 | Linux           |      1
 3 | Ubuntu          |      2
 4 | Kubuntu         |      3
 5 | Database        |
 6 | Operating Systems |
(6 rows)
```

You can either close the connection and start it over to get a regularly setup version of `search_path`, or set it manually in your current connection by means of `set_config()`, for example:

```
forumdb_test=> SELECT pg_catalog.set_config('search_path', 'public,
"user", false);
set_config
```

```

-----
public, "$user"
(1 row)

forumdb_test=> SELECT * FROM tags;
pk |      tag      | parent
---+-----+-----
 1 | Operating Systems |
 2 | Linux           |      1
 3 | Ubuntu          |      2
 4 | Kubuntu         |      3
 5 | Database        |
 6 | Operating Systems |
(6 rows)

```

As you can see, now, the connection works just fine.

It is also possible to perform a backup (and a restore) in the very same database. First of all, `pg_dump` must include a special option called `--create`, which instructs the application to issue `CREATE DATABASE` as the very first instruction for the restoration:

```

$ pg_dump --column-inserts --create -f backup_forumdb.sql forumdb
$ less backup_forumdb.sql
...
CREATE DATABASE forumdb WITH TEMPLATE = template0 ENCODING = 'UTF8'
LOCALE_PROVIDER = libc LOCALE = 'en_US.utf8';

ALTER DATABASE forumdb OWNER TO forum;

\connect forumdb

...

```

As you can see, the output of `pg_dump` now includes the creation of the database, as well as the special `\c` command to connect immediately to such a database. In other words, launching this file through `psql` will restore the full content in the right database when the latter does not exist.

In order to test this, let's destroy our beloved database and restore it by means of initially connecting to template1:

```
$ psql -c 'DROP DATABASE forumdb';
DROP DATABASE
$ psql
postgres=# \i backup_forumdb.sql
...
forumdb=#
```

Note how the command prompt has changed to reflect the fact that we are now connected to the restored forumdb database.

So, which version of dump and restoration should you use? If you replicate the database in another cluster, for example, to migrate a staging database to production, you should include the `--create` option to let the database engine create the database for you. If you migrate the database content to an existing database, then the `--create` option must not be present at all because there is no need to set up a database; this can be risky because you could restore objects to the wrong database, so you need to carefully check that you are connected to the right database before reloading the backup script.

If you migrate the content of the database to another engine, such as another relational database, you should use options such as `--inserts` or `--column-inserts` to make the database backup more portable.

Limiting the amount of data to backup

`pg_dump` allows an extensive set of filters and flags to be used to limit the amount of data to back up. For example, you could decide to dump only the database schema without any data in it, and this can be achieved by means of the `-s` flag. On the other hand, you could already have the database schema in place, and you may only need the database content without any DDL statement. This can be achieved with the `-a` option. You can, of course, combine different `pg_dump` commands to get separate backups:

```
$ pg_dump -s -f database_structure.sql forumdb
$ pg_dump -a -f database_content.sql forumdb
```

You will end up with a file called `database_structure.sql` that contains all the different `CREATE TABLE` statements, and another file that contains only the `COPY` (or `INSERT` statements if you specified the `--inserts`) statements.